

Mutex as Shared Resource

STM32 FreeRTOS

SSD1306 OLED

I2C + Scheduler

Author	Siva Krishna Donthineni
Board	STM32 Nucleo-F103RB (ARM Cortex-M3 @ 72 MHz)
Display	SSD1306 OLED 0.96 inch — I2C at 100 KHz
RTOS	FreeRTOS CMSIS V1
Tools	STM32CubeIDE 2.1.1 CubeMX PuTTY
GitHub	github.com/DONTHINENISIVAKRISHNA

This document is my personal account of building Project-3 from scratch on real hardware. I am writing this so I can look back and understand exactly what I did, what went wrong, how I fixed it, and what I learned. Every error listed here is something I actually hit. Every fix is something I actually applied. Nothing is copied from a tutorial.

Contents

1	What Is This Project	6	Code Logic — How It All Works
2	Hardware I Used	7	Final Output
3	Software and Tools	8	What I Learned
4	Step by Step — What I Did	9	Real World Applications
5	Errors I Got and How I Fixed Them	10	GitHub

1. WHAT IS THIS PROJECT

This project is about protecting a shared hardware resource using a Mutex in FreeRTOS. I have one SSD1306 OLED display connected to my STM32 Nucleo board over I2C. I wrote two FreeRTOS tasks that both need to use this display. Without protection, the FreeRTOS scheduler can switch between tasks in the middle of an I2C write, causing corrupted output on the screen. A Mutex solves this by ensuring only one task touches the display at a time.

In simple terms:

- One OLED display. Two tasks. One lock.
- DisplayTask writes Counter and Uptime to OLED every 500ms.
- CounterTask increments a counter in RAM every 1000ms.
- A Mutex ensures they never touch the shared I2C bus at the same time.
- FreeRTOS scheduler runs both tasks concurrently on one CPU.

2. HARDWARE I USED

Component	What I used it for
STM32 Nucleo-F103RB	Main board. ARM Cortex-M3 at 72 MHz. Runs FreeRTOS and both tasks. Has onboard ST-Link so no external programmer needed.
SSD1306 OLED 0.96 inch	The shared display. 128x64 pixels over I2C. This is the shared resource protected by Mutex.
USB Cable (Mini-USB)	One cable handles everything — flashing firmware, powering board, and serial monitor for PuTTY.
Jumper Wires (Female-Male)	4 wires connecting OLED to Nucleo Arduino headers D14 and D15.
Breadboard	Held the OLED module stable during wiring.
PC with Windows	Ran STM32CubeIDE, CubeMX, and PuTTY.

3. SOFTWARE AND TOOLS

Tool	What I used it for
STM32CubeMX	Configured all peripherals graphically — I2C, USART, FreeRTOS tasks. Generated base project code.
STM32CubeIDE 2.1.1	Wrote all C code. Compiled with ARM GCC. Flashing firmware to board. Debugged all errors.
PuTTY	Serial terminal. Opened COM5 at 115200 baud. Showed real time printf output from both tasks.
SSD1306 Library	Open source library by afiskon on GitHub. Handles all I2C communication with the OLED.

GitHub	Uploaded complete project for version control and portfolio.
--------	--

■ 4. STEP BY STEP — WHAT I DID

Step 1 — Created CubeMX Project

Opened STM32CubeMX and created a new project for STM32F103RBTx (Nucleo-F103RB). Configured all peripherals one by one:

- System Core → SYS → Debug: Serial Wire
- System Core → RCC → HSE: Crystal/Ceramic Resonator
- System Core → SYS → Timebase Source: TIM1 (FreeRTOS needs SysTick free)
- Connectivity → I2C1 → Mode: I2C, Standard Mode 100 KHz
- Connectivity → USART2 → Mode: Asynchronous, 115200 baud
- Middleware → FREERTOS → Interface: CMSIS_V1
- FREERTOS Tasks: DisplayTask (stack 512, Normal priority) and CounterTask (stack 128, Normal priority)
- Clock Configuration → HCLK set to 72 MHz

Note: I initially configured USART1 instead of USART2. Nucleo-F103RB virtual COM port uses USART2 — not USART1. Fixed this later.

Step 2 — Added SSD1306 Library

Downloaded from github.com/afiskon/stm32-ssd1306. Copied 5 files:

```
Core/Inc/ <--  ssd1306.h
Core/Inc/ <--  ssd1306_fonts.h
Core/Inc/ <--  ssd1306_conf.h    (renamed from ssd1306_conf_template.h)
Core/Src/ <--  ssd1306.c
Core/Src/ <--  ssd1306_fonts.c
```

Note: ssd1306_conf_template.h must be renamed to ssd1306_conf.h otherwise library cannot find its config file.

Step 3 — Wrote main.c Additions

Block A — Include stdio

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */
```

Block B — Redirect printf to UART2

```
/* USER CODE BEGIN 0 */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
/* USER CODE END 0 */
```

Block C — Startup banner in USER CODE BEGIN 2

```
HAL_Delay(100);
printf("=====\r\n");
printf("  Project-3: Mutex as Shared Resource  \r\n");
printf("  OLED + FreeRTOS Scheduler              \r\n");
printf("=====\r\n");
```

Block D — Create Mutex before scheduler in USER CODE BEGIN RTOS_THREADS

```
extern SemaphoreHandle_t oledMutex;
```

```
oledMutex = xSemaphoreCreateMutex();
if (oledMutex == NULL) {
    printf("[MAIN] FATAL: Mutex creation failed!\r\n");
    Error_Handler();
}
printf("[MAIN] Mutex created OK. Scheduler starting...\r\n");
```

Note: Mutex must be created before osKernelStart(). If created inside a task, other task might try to take it before it exists — NULL pointer crash.

Step 4 — Wrote freertos.c

Includes and Global Variables

```
#include <stdio.h>
#include <string.h>
#include "semphr.h"
#include "ssd1306.h"
#include "ssd1306_fonts.h"

extern I2C_HandleTypeDef hi2c1;
SemaphoreHandle_t oledMutex = NULL;
volatile uint32_t g_counter = 0;
```

DisplayTask

```
void StartDisplayTask(void const * argument) {
    if (oledMutex == NULL) { vTaskDelete(NULL); return; }
    ssd1306_Init();
    ssd1306_Fill(Black);
    ssd1306_UpdateScreen();
    printf("[DISPLAY] OLED initialized.\r\n");
    char line1[20], line2[20];
    for (;;) {
        if (xSemaphoreTake(oledMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
            ssd1306_Fill(Black);
            snprintf(line1, sizeof(line1), "Counter: %lu", (unsigned long)g_counter);
            ssd1306_SetCursor(0, 0);
            ssd1306_WriteString(line1, Font_7x10, White);
            uint32_t uptime = xTaskGetTickCount() / 1000;
            snprintf(line2, sizeof(line2), "Up: %lus", (unsigned long)uptime);
            ssd1306_SetCursor(0, 20);
            ssd1306_WriteString(line2, Font_7x10, White);
            ssd1306_SetCursor(0, 40);
            ssd1306_WriteString("Project-3 FreeRTOS", Font_6x8, White);
            ssd1306_UpdateScreen();
            printf("[DISPLAY] Counter: %lu | Uptime: %lus\r\n",
                (unsigned long)g_counter, (unsigned long)uptime);
            xSemaphoreGive(oledMutex); // give BEFORE delay
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

CounterTask

```
void StartCounterTask(void const * argument) {
    printf("[COUNTER] Started. Incrementing every 1000ms.\r\n");
```

```
for (;;) {
    if (oledMutex != NULL) {
        if (xSemaphoreTake(oledMutex, pdMS_TO_TICKS(200)) == pdTRUE) {
            g_counter++;
            printf("[COUNTER] Count: %lu\r\n", (unsigned long)g_counter);
            xSemaphoreGive(oledMutex);
        }
    }
    vTaskDelay(pdMS_TO_TICKS(1000));
}
}
```

■ 5. ERRORS I GOT AND HOW I FIXED THEM

Every error below is something I actually hit during this project. Writing them down so I remember the diagnosis and fix for future projects.

Error 1 — `ssd1306_conf.h`: No such file or directory

fatal error: `ssd1306_conf.h`: No such file or directory

Cause: Downloaded file was named `ssd1306_conf_template.h`. Library looks for `ssd1306_conf.h` specifically.

Fix: Renamed `ssd1306_conf_template.h` to `ssd1306_conf.h` in `Core/Inc/`

✓ **Rename the template file — do not leave it as `ssd1306_conf_template.h`**

Error 2 — `stm32f0xx_hal.h`: No such file or directory

fatal error: `stm32f0xx_hal.h`: No such file or directory

Cause: `ssd1306_conf.h` had STM32F0 defined by default. My board is STM32F1.

Fix: Changed in `ssd1306_conf.h`:

```
///#define STM32F0    <-- commented out
#define STM32F1    <-- uncommented
```

✓ **Always check `ssd1306_conf.h` and set the correct STM32 family**

Error 3 — `SemaphoreHandle_t` unknown type

error: unknown type name '`SemaphoreHandle_t`'

error: implicit declaration of '`xSemaphoreTake`'

Cause: `semphr.h` not included in `freertos.c`. Mutex functions live in that header.

Fix: Added to USER CODE BEGIN Includes in `freertos.c`:

```
#include "semphr.h"
```

✓ **Always include `semphr.h` when using Mutex or Semaphore functions**

Error 4 — Multiple definition of `StartDisplayTask`

error: multiple definition of '`StartDisplayTask`'

error: multiple definition of '`StartCounterTask`'

Cause: CubeMX auto-generated empty versions of both task functions at the bottom of `main.c`. I also wrote real implementations in `freertos.c`. Linker found two definitions of same function.

Fix: Deleted the two auto-generated empty functions from bottom of `main.c`. Kept only the real implementations in `freertos.c`.

✓ **Delete CubeMX auto-generated empty task functions from `main.c`**

Error 5 — Multiple definition of `SystemCoreClock`

multiple definition of '`SystemCoreClock`'

multiple definition of '`SystemInit`'

Cause: CubeIDE Import tool copied all files from `Core/Src` including `system_stm32f1xx.c` which already existed in `Drivers/CMSIS`. Two copies of same file.

Fix: Right clicked `system_stm32f1xx.c` in Application/User/Core and deleted it. Original in Drivers/CMSIS stays untouched.

✓ **When importing files, only import `ssd1306.c` and `ssd1306_fonts.c` — nothing else**

Error 6 — PuTTY showing nothing

Cause: I used USART1 in CubeMX and `huart1` in `__io_putchar()`. Nucleo-F103RB virtual COM port is hardwired to USART2 (PA2/PA3). USART1 TX output goes nowhere on this board.

Fix: Disabled USART1 in CubeMX. Enabled USART2 Asynchronous. Changed `huart1` to `huart2` in `__io_putchar()` in `main.c`.

✓ **Nucleo-F103RB always use USART2 for virtual COM port — not USART1**

Error 7 — OLED completely dark, `ssd1306_Init()` hanging forever

Cause: I2C1 was on PB6/PB7 in CubeMX but I could not find those exact physical pins on the CN7 morpho header. `ssd1306_Init()` sent I2C commands and waited for ACK that never came — function hung indefinitely.

How I diagnosed it: Added `printf` before and after `ssd1306_Init()` to confirm where it was hanging. Then wrote an I2C bus scanner:

```
printf("[DISPLAY] Scanning I2C bus...\r\n");
for (uint8_t addr = 1; addr < 128; addr++) {
    if (HAL_I2C_IsDeviceReady(&hi2c1, (addr<<1), 1, 10) == HAL_OK)
        printf("Found device at 0x%02X\r\n", addr);
}
printf("[DISPLAY] Scan complete.\r\n");
```

Scanner found nothing — wiring problem confirmed.

Fix: Remapped I2C1 from PB6/PB7 to PB8/PB9 in CubeMX. PB8/PB9 map to Arduino headers D15 (SCL) and D14 (SDA) which are clearly labeled on the Nucleo board. Connected OLED wires there. Scanner immediately found device at 0x3C.

✓ **On Nucleo-F103RB use D15 for SCL and D14 for SDA — clearly labeled Arduino headers**

■ 6. CODE LOGIC — HOW IT ALL WORKS

6.1 Why Mutex Must Be Created Before the Scheduler

```
main() {
    MX_I2C1_Init();
    MX_USART2_Init();

    oledMutex = xSemaphoreCreateMutex(); // created HERE
    // Mutex exists before any task runs

    osKernelStart(); // scheduler starts -- never returns
}

// If created inside DisplayTask instead:
// CounterTask might call xSemaphoreTake(oledMutex)
// before DisplayTask creates it
// oledMutex = NULL --> HardFault crash
```

6.2 How the Scheduler Runs Both Tasks

```
SysTick fires every 1ms
    Scheduler wakes up
    Checks all tasks:
    +-- DisplayTask READY? --> run it
    +-- CounterTask READY? --> run it
    +-- Task BLOCKED?      --> skip, leave sleeping

vTaskDelay(500ms) --> DisplayTask moves to BLOCKED
                    Scheduler immediately runs CounterTask
                    No CPU wasted waiting

After 500ms --> DisplayTask moves back to READY
                Scheduler runs it again next tick
```

6.3 Mutex Execution Flow at Runtime

```
T=500ms DisplayTask wakes up
        xSemaphoreTake(oledMutex) --> ACQUIRED

CounterTask also wakes up
xSemaphoreTake(oledMutex) --> BLOCKED (waiting)

DisplayTask writes to OLED via I2C
ssd1306_UpdateScreen() completes fully
xSemaphoreGive(oledMutex) --> RELEASED

CounterTask wakes up immediately
xSemaphoreTake(oledMutex) --> ACQUIRED
g_counter++
xSemaphoreGive(oledMutex) --> RELEASED

Both tasks sleep. Cycle repeats forever.
```

6.4 Critical Rule — Give Mutex Before Sleeping

WRONG -- holding Mutex during sleep:

```
xSemaphoreTake(oledMutex)
// write to OLED
vTaskDelay(500ms)          // sleeping while holding lock
xSemaphoreGive(oledMutex) // CounterTask blocked 500ms unnecessarily
```

CORRECT -- give before sleep:

```
xSemaphoreTake(oledMutex)
// write to OLED
xSemaphoreGive(oledMutex) // release immediately after work
vTaskDelay(500ms)        // sleep without holding lock
```

6.5 Why g_counter Is Declared volatile

```
volatile uint32_t g_counter = 0;
// volatile = always read from RAM
// never cache in CPU register
// because CounterTask can change it at any time
// without volatile, compiler might use stale cached value
```

■ 7. FINAL OUTPUT

7.1 OLED Display

```
+-----+
| Counter: 5      | CounterTask increments every 1s
|                |
| Up: 6s          | Uptime since boot
|                |
| Project-3 FreeRTOS | Fixed label
+-----+
Refreshes every 500ms. No corruption. Mutex working.
```

7.2 PuTTY Output

```
=====
Project-3: Mutex as Shared Resource
OLED + FreeRTOS Scheduler
=====
[MAIN] Mutex created OK. Scheduler starting...
[COUNTER] Started. Incrementing every 1000ms.
[DISPLAY] OLED initialized.
[DISPLAY] Counter: 0 | Uptime: 0s
[COUNTER] Count: 1
[DISPLAY] Counter: 1 | Uptime: 1s
[COUNTER] Count: 2
[DISPLAY] Counter: 2 | Uptime: 2s
...

```

7.3 Observation — Uptime 1 Second Ahead of Counter

I noticed Uptime was always 1 second ahead of Counter. The reason: `ssd1306_Init()` takes approximately 1 second to initialize the OLED over I2C. During that time the FreeRTOS tick counter was already running. So when DisplayTask entered its loop, Uptime was already at 1 second while Counter was still at 0. This is not a bug. It proves both the tick counter and tasks are working independently and correctly.

■ 8. WHAT I LEARNED

Concept	What I understand now
Mutex	A kernel lock. One task holds it at a time. Other tasks BLOCKED until released. Protects shared resources from race conditions.
Race Condition	Without Mutex, scheduler switches mid I2C write. Two tasks corrupt shared bus. OLED freezes or shows garbage.
FreeRTOS Scheduler	Runs every 1ms via SysTick. Switches READY tasks. Leaves BLOCKED tasks sleeping. Zero CPU waste.
BLOCKED State	Task waiting for Mutex or delay. Consumes zero CPU. Scheduler ignores it until READY.

Nucleo Virtual COM	Nucleo-F103RB uses USART2 for USB serial. Not USART1. Must configure USART2 in CubeMX.
I2C Scanner	Loop from 1 to 127 calling HAL_I2C_IsDeviceReady(). Found OLED at 0x3C after fixing pins.
volatile keyword	Tells compiler variable can change from another task. Always read from RAM not cached register.
USER CODE blocks	CubeMX only preserves code inside USER CODE BEGIN/END. Never write outside these blocks.

■ 9. REAL WORLD APPLICATIONS

Industry	How this concept is used
Automotive ECUs	Multiple tasks sharing CAN bus and SPI sensors. Mutex protects every shared peripheral.
Medical Devices	Ventilators run multiple FreeRTOS tasks. Mutex ensures sensor data is not corrupted between read and display tasks.
Industrial HMI	Factory panels with multiple tasks updating one display. Exact same pattern as this project.
Drone Controllers	IMU, GPS, motor tasks sharing SPI bus. Mutex prevents mid-read interruptions.
IoT Devices	WiFi, sensor, display tasks sharing hardware. FreeRTOS scheduler manages all concurrently.
5G Base Stations	Protocol tasks sharing radio hardware. Mutex and semaphore synchronisation at every layer.

■ 10. GITHUB

Complete project uploaded to GitHub for portfolio and reference.

Repository : github.com/DONTHINENISIVAKRISHNA
Project : STM32-FreRTOS-Project3-Mutex-OLED

Files:

```

Core/Src/main.c          -- Mutex creation, UART redirect, startup banner
Core/Src/freertos.c      -- DisplayTask and CounterTask
Core/Inc/ssd1306_conf.h  -- Library config for STM32F1 I2C
Core/Inc/ssd1306.h       -- OLED library header
Core/Src/ssd1306.c       -- OLED library implementation
MutexOLED.ioc           -- CubeMX project file
README.md               -- Wiring guide and project description

```